

Lab 2: Sensor-Driven Machine Learning — Weather Station or Keyword Spotting

last update: 6/8/26

General Information

Lab Mode: Remote. Work on the lab remotely using your own computer and lab dev kit.

Lab Due Date: Posted in Canvas. Submit all the required deliverables by the due date.

Lab Report Submission: Submit via Canvas. No email submissions are accepted.

Lab Contents: Additional requirements applied for EECE.5862. Details are below.

Choose-one-track lab. This lab offers **two tracks** — **Track W (Weather Station)** and **Track K (Keyword Spotting)**. **Pick exactly one.** Both are worth the same points and graded against the same rubric; they differ only in the on-board sensor, the dataset, and the training tool. State your chosen track on the title page of your report.

Lab Description

Requirements for both EECE.4862 and EECE.5862 students

Lab 1 deployed a model that someone else trained. Lab 2 closes the full loop: you collect data **from a real on-board sensor of the Arduino Nano 33 BLE Sense Rev2**, train (or tune) a model on that kind of data **off-device**, quantize it to **8-bit**, deploy it back onto the board, and watch it **act** on live sensor input — the complete *sense* → *infer* → *act* pipeline. The lab has two parts: **Part A — Sense** and **Part B — Infer + Act**.

The two tracks at a glance:

	Track W — Weather Station	Track K — Keyword Spotting
Sensor	On-board HS3003 temperature/humidity	On-board MP34DT06 PDM microphone
Task	Predict snowfall (yes/no) from the last 3 hours of temperature + humidity	Recognize two spoken color keywords and blink the matching RGB LED
Train with	TensorFlow in a provided Colab notebook → TFLite Micro	Edge Impulse (browser) → Arduino library
Cookbook	Chapter 3	Chapter 4

Pick one track and complete the following requirements:

- (1) **Choose your track** and state it on your report title page. Both tracks run entirely on **on-board** peripherals — no breadboard wiring is required for the core lab. Track W exposes the full TensorFlow → TFLite → C-array → TFLM path end-to-end; Track K hands the training pipeline to Edge Impulse and focuses on data collection and the audio feature pipeline. Commit to one track for the whole lab; do not mix.
- (2) **Part A.1 — Read the sensor and stream it over Serial.** *Track W:* starting from `starter/w1-th-capture/`, initialize the HS3003 via `Arduino_HS300x`, then once per second read temperature (°C) and relative humidity (%) and print them as one **CSV line** (e.g. `millis(),temp_c,humidity_pct`) at 9600 baud. *Track K:* starting from `starter/k1-audio-capture/`, initialize the on-board PDM microphone, capture a buffer of audio samples, and report over Serial that audio is being captured (samples read and a simple level/amplitude per buffer). This is the sketch you will later point the **Edge Impulse data forwarder** at. **Deliverable:** the sketch and a Serial Monitor screenshot of live values (≥ 10 lines for Track W; several capture reports for Track K).
- (3) **Part A.2 — From raw samples to model features.** In your report, explain ($\frac{1}{2}$ –1 page, with a diagram) how the raw stream becomes the model's **input tensor**. *Track W:* the model predicts snow from the **last 3 hours** of temperature and humidity, not a single reading — describe what one feature vector contains (how many readings, over what span, at what interval) and why a *window* of history beats a single sample; sketch how your A.1 capture would buffer readings into that window. *Track K:* describe the **MFE (Mel-filterbank energy)** feature pipeline Edge Impulse uses — what an MFE/spectrogram represents, why a frequency-domain representation suits speech, and the shape (time $\times$ frequency bins) of the feature block the CNN sees as its input “image.”
- (4) **Part B.1 — Train or tune the model off-device.** *Track W:* open the **provided notebook** in `training/` in Google Colab and **run it top to bottom**. It loads the bundled weather dataset, builds the temperature/humidity feature window, trains a small snowfall classifier, **quantizes it to int8**, and exports the model as a C array (`model.cpp`). Make the **one required change** the notebook asks for (a different window length or train/val split) so your report reflects a run you drove; record the train/validation accuracy. *Track K:* in **Edge Impulse**, build an **MFE + CNN** impulse to recognize **two color keywords** — two of red / green / blue — plus the background classes noise and unknown. Collect $\geq \sim 30$ **one-second utterances per color keyword** (use your A.1 sketch with the `edge-impulse-data-forwarder`, or the EI phone app); for noise/unknown you may use **Edge Impulse's prebuilt datasets**. Train and record the validation accuracy and confusion matrix. **Deliverable:** *Track W* — your notebook run (accuracy + the change you made) and exported `model.cpp`; *Track K* — a screenshot of your EI **Model** page (accuracy + confusion matrix) and the impulse design (MFE feature block + CNN block).
- (5) **Part B.2 — Deploy and run on the Nano.** *Track W:* start from `starter/w2-weather-inference/` and drop in the `model.cpp` your notebook exported. The scaffold provides the TFLM interpreter setup, the rolling **feature-window buffer**, and the **int8 quantization helper**; you read live temperature/humidity, push each reading into the window, call `Invoke()`, and **act** (light the on-board LED and print SNOW LIKELY / NO SNOW). The window-length constant must match the notebook. (*The scaffold samples on an accelerated cadence — set in its folder README — so the window fills in a minute or two for the demo.*) *Track K:* export your trained EI project as an **Arduino library** (.zip), add it via **Sketch → Include Library → Add .ZIP Library...**, and start from `starter/k2-kws-inference/`, a thin wrapper

around the EI-generated `run_classifier()` continuous-inference loop. Complete the `// TODO(student)` blocks so a **confidently** recognized keyword blinks the on-board RGB LED **once in the matching color** (red $\rightarrow$ red, etc.), and does nothing for `noise / unknown` or low-confidence results. **Stop the data forwarder/daemon before uploading**, or the upload fails on a busy serial port. **Deliverable:** the completed sketch(es) and a short **video or stitched screenshots** of the board responding to live input.

- (6) **Part B.3 — Measure and reflect.** Report a small table with **accuracy** (Track W: test accuracy from B.1 + a note on live behavior; Track K: EI validation/test accuracy + how reliably it responds to your live speech), **flash usage** and **RAM usage** (from the Arduino IDE compile output), and **inference latency** (Track W: time `Invoke()` with `micros()`, **min/mean/max** over ≥ 100 inferences; Track K: read EI's per-inference **DSP + classification** timing). Then write **two short paragraphs** (3–5 sentences each): (a) compare this model's flash/RAM footprint to Lab 1's hello-world model — where did the growth come from (model, feature pipeline, or runtime)? (b) what was the **hardest part of getting real data to work**, and what would you change about your data collection next time? **Deliverable:** the table, the serial/EI log backing your latency numbers, and the two paragraphs.

Note: (1) Begin each part with a brief *model* of the system — for Part A, a sketch of the data $\rightarrow$ feature transform; for Part B, a block diagram of the *sense $\rightarrow$ infer $\rightarrow$ act* loop. (2) Use the **Arduino C/C++** programming model (`setup()` / `loop()`, libraries via the IDE). Skeleton starter sketches (file structure + `// TODO(student)` hints) for the sketches above are in the lab repository at <https://github.com/ACANETS/eece-4862-5862-labs> under `lab-2/starter/`. The skeletons compile/upload as-is so you can confirm your toolchain, but credit comes from your implemented logic: a starter submitted with its TODOs unfilled earns no credit for that part. (3) Your **Rev2** board uses the **HS3003** temperature/humidity sensor (Arduino_HS300x); wherever the *TinyML Cookbook* shows the Rev1 Arduino_HTS221, substitute accordingly.

Additional requirements for EECE.5862 students

5862 students must complete **one** of the three options below (4862 students may attempt one for up to +2 bonus credit, capped at the lab's 20-point maximum).

- (7) **Option C.1 — Quantization cost (cross-track).** Compare your model **before and after int8 quantization** (Track W: the float32 TFLite model vs. the int8 model; Track K: Edge Impulse's *unoptimized (float32)* vs. *quantized (int8)* deployment). Report, for each, the **accuracy**, **flash size**, and **inference latency**. Two paragraphs: what did quantization cost you in accuracy, and what did it buy you in size and speed?
- (8) **Option C.2 — Track W: add a third feature.** The Rev2 board also carries an **LPS22HB barometric pressure sensor** (Arduino_LPS22HB). Add barometric pressure as a third input feature, retrain, and compare snowfall-prediction accuracy with and without it. Two paragraphs: did pressure help, and does that match your physical intuition about snow?
- (9) **Option C.3 — Track K: robustness characterization.** Characterize your keyword model's **false-accept / false-reject** behavior. Speak each keyword 20 times and tally correct detections; then speak 20 *non*-keyword words and tally spurious triggers. Build a small confusion matrix from live on-device behavior (not just EI's validation set). Two paragraphs: where does it fail, and how would more data or the EON Tuner change that?

Hints:

Skeleton sketches with `// TODO(student)` markers are under `lab-2/starter/`; build one unmodified first to confirm your toolchain, then fill the TODOs. *Track W*: the provided Colab notebook in `lab-2/training/` is **self-contained** — it bundles the dataset, so you need **no weather-API key** and no external download; running it top to bottom prints the window length and normalization constants your sketch must match. *Track K*: two keywords keeps data collection near Lab 1’s effort level; the EON Tuner is **optional** (it is a Part C deepening option, not required for the core lab). Collect from more than one speaker if you can — single-speaker datasets generalize poorly.

Lab Materials:

Required Parts:

Part name	Quantity	Notes
Arduino Nano 33 BLE Sense Rev2 (SKU ABX00070)	1	with headers; on-board HS3003, PDM mic, and RGB LED
USB-A to micro-USB cable	1	must be data -capable, not power-only

No breadboard or external components are required for the core lab — both tracks use **on-board** sensors and LEDs. (Option C.2 uses the on-board LPS22HB pressure sensor, also requiring no extra hardware.)

Required Software:

- (a) Arduino IDE 2.x (<https://www.arduino.cc/en/software>) with the **Arduino Mbed OS Nano Boards** package (from Lab 1).

Track W:

- (b) `Arduino_HS300x` library (on-board temperature/humidity sensor) — install via the IDE Library Manager.
- (c) `Arduino_TensorFlowLite` (TFLM runtime) — the same `Arduino_TensorFlowLite.zip` from Lab 1 (`../lab-1/Arduino_TensorFlowLite.zip` in the lab repo), added via **Sketch** → **Include Library** → **Add .ZIP Library**....
- (d) Google Colab (browser, no install) to run the provided training notebook in `lab-2/training/`.

Track K:

- (e) An **Edge Impulse** account (free) — <https://edgeimpulse.com>.
- (f) The **Edge Impulse CLI** (for the data forwarder / serial data collection), or the Edge Impulse phone app.
- (g) The Edge Impulse-generated **Arduino library**, which you export from your project and add via **Sketch** → **Include Library** → **Add .ZIP Library**....

Lab Instructions

Step 1:

Choose your track (W or K) and install its software. Both tracks need the Arduino IDE 2.x and the Mbed OS Nano Boards package from Lab 1. Track W: install `Arduino_HS300x` and confirm the `Arduino_TensorFlowLite` library from Lab 1. Track K: create a free Edge Impulse account and install the Edge Impulse CLI.

Step 2:

Complete **Part A.1** (requirement (2)): capture your sensor and stream it over Serial from the `w1/k1` starter. Open the Serial Monitor at 9600 baud and confirm live values stream.

Step 3:

Complete **Part A.2** (requirement (3)): write the feature-pipeline explanation and diagram for your track.

Step 4:

Complete **Part B.1** (requirement (4)): Track W — run the provided Colab notebook top to bottom, make the required change, and export `model.cpp`. Track K — collect your dataset with the EI data forwarder (or phone app), build the MFE + CNN impulse, and train; record accuracy and the confusion matrix.

Step 5:

Complete **Part B.2** (requirement (5)): deploy to the Nano from the `w2/k2` starter and wire up the **act** stage (LED on snowfall prediction, or matching-color blink on a recognized keyword). Capture a short video or screenshots of the live response.

Step 6:

Complete **Part B.3** (requirement (6)): record the accuracy / flash / RAM / latency table and write the two reflection paragraphs. Print `unsigned long` timing with `Serial.print()` (not `MicroPrintf %lu`), and remember the nRF52840's `micros()` has $\sim 4\ \mu\text{s}$ resolution, so the **mean** over many inferences is the meaningful number.

EECE5862:

<For 5862 students>: Complete one of options C.1, C.2, or C.3 from requirements (7)–(9). Document your method, results, and analysis paragraphs in the lab report.

Step 7:

Test and debug your design. Write your lab report following the **Lab Report Template** posted on Canvas. Include all required deliverables: your chosen track on the title page, the feature diagram, training/accuracy evidence (notebook run or EI Model page), the measurement table, serial/EI logs, screenshots, and code listings (screenshots of key snippets — do not paste full dumps), plus your reflection paragraphs. Per the syllabus **AI Use Policy**, disclose any AI tool you used (tool, prompts, output, and modifications).

Step 8:

Push your source code to a **private** GitHub repository. Submit the lab report PDF and a `.zip` of your Arduino sketches via Canvas. Track W: also include your run of the notebook (`.ipynb` with output cells) and your exported `model.cpp`. Track K: also include a link to your public Edge Impulse project (or the exported library zip).

Step 9:

Record a short demo video showing your working setup (temperature/humidity change driving the snowfall prediction and LED for Track W, or spoken keywords blinking the matching-color LED for Track K) and post it on YouTube. Include the video URL in your lab report.

###

Please refer to lab demonstration and report guidelines for demonstration and report writing.